

Benchmark Report — FinReflectKG on ArangoDB

Status: v3.0 · 2026-07-23 · **cross-distribution** (baseline + OneShard + SmartGraph), now with **Class 8** — **label-rooted aggregations** enabled by the type-leading indexes added 2026-07-22. All three distributions carry an identical index set (§4.5 parity). **Suite:** [scripts/benchmark.py](#) (multi-db + explain-locality) · raw: [data/benchmark_cross.json](#) **Related:** [load-report.md](#) · [PRD.md](#) §4.2, §6 · [multi-distribution-plan.md](#) · [nl-graphrag.md](#)

Method

Each query: 1 warmup + 7 timed iterations; we report **median** latency plus the **deterministic profile counters** (`scannedIndex` / `scannedFull` / `filtered`) and **explain locality** (`RemoteNode` = cross-DBServer hops). Latency on the shared remote cluster is noisy (a single query has ranged 0.2–20 s), so the scanned-edge counts and `RemoteNode` counts are the portable signal; treat wall-clock as *indicative*. Typed 1-hop queries (2–4a, 8) are written as **direct edge-collection queries** — the access path that engages the vertex-centric / type-leading indexes (pattern traversals use the generic edge index; see [load-report.md](#)).

Indexes on `relations` (identical across all three DBs): `edge(_from,_to)` , node-anchored VCI's `vci_from_type_totype(_from,type,_toType)` / `vci_to_type_fromtype(_to,type,_fromType)` , `rel_ticker_year(ticker,year)` , and the type-leading `vci_type_fromtype_totype(type,_fromType,_toType)` / `vci_type_totype_fromtype(type,_toType,_fromType)` .

Results — cross-distribution (median ms / `scannedIndex` / `RemoteNode`)

Entities are **discovered per database** because the SmartGraph rewrites keys with a `ticker` prefix and duplicates shared concepts per company, so a "supernode" on the smart db is a *specific company's copy*, not the single global one.

#	Query	class	Baseline	OneShard	SmartGraph
1	point lookup by <code>name</code>	lookup	0.6ms / 28 / r1	0.6ms / 28 / r1	4.0ms / 873 / r1
2	1-hop typed OUT (<code>operates_in</code> → GPE)	VCI (from)	13.7ms / 6,290 / r1	14.3ms / 6,290 / r1	14.6ms / 6,290 / r1
3	reverse typed IN (<code>has_stake_in</code>)	VCI (to)	14.8ms / 2,882 / r1	14.7ms / 2,882 / r1	0.5ms / 7 / r1
4a	supernode pruned (<code>net income</code>)	VCI (to)	200.3ms / 59,315 / r1	201.5ms / 59,315 / r1	0.9ms / 77 / r1
4b	supernode unpruned (traversal)	edge idx	404.0ms / 99,295 / r0	361.7ms / 99,295 / r1	1.4ms / 136 / r2
5	2-hop (<code>company</code> → <code>metric</code> → <code>peers</code>)	traversal	23.7ms / 2,747 / r0	2.0ms / 2,026 / r1	7.5ms / 1,099 / r4
6	temporal slice (<code>ticker</code> + year)	rel_ticker_year	17.9ms / 10,385 / r1	17.4ms / 10,385 / r1	0.4ms / 0 / r1 †
7	NL-grounding (<code>edge</code> → <code>chunks</code>)	edge + PK	4.9ms / 20 / r1	0.8ms / 20 / r1	4.4ms / 20 / r1
8a	label agg: orgs by #GPE (<code>operates_in</code>)	type idx (from)	710ms / 313,407 / r1	705ms / 313,407 / r1	602ms / 313,407 / r1
8b	label agg: GPEs by #org (<code>operates_in</code> rev)	type idx (to)	737ms / 313,407 / r1	735ms / 313,407 / r1	726ms / 313,407 / r1
8c	label agg: orgs in litigation (<code>involved_in</code>)	type idx (from)	20.5ms / 5,780 / r1	20.3ms / 5,780 / r1	19.4ms / 5,780 / r1

`scannedFull` = 0 on every index-backed query (1–4a, 6, 8) across all three DBs.

Findings

1. **The VCIs narrow perfectly** (queries 2, 3, 4a, 6): the index returns exactly the matching edges — `scannedFull` = 0, `filtered` = 0. Core result the model was built for.

2. **SmartGraph decomposes supernodes** (4a/4b). Design 2 duplicates each shared concept per company, so a per-company query scans **77–136 edges instead of 59 K–99 K**: 4a drops **200.3 ms** → **0.9 ms** and 4b **404.0 ms** → **1.4 ms**. Company-scoped analytics never touch other companies' data. Reverse-typed (3) similarly drops to **0.5 ms / 7 edges**.
3. **NEW — Class 8 label-rooted aggregations are now feasible**. "All `:ORG` that ..." has no bound start node, so the node-anchored VCI's can't serve it; before the type-leading indexes such queries scanned the full 17.5 M-edge collection and **timed out**. The `type`-leading indexes prune to the matching slice: **8c** ("orgs involved in litigation", 5,780 edges = 0.03 %) runs in **~20 ms**; **8a/8b** (313,407 edges = 1.79 %) in **~0.7 s**. For contrast, the *pattern-traversal* form of 8c (rooted at all 22,640 `:ORG` nodes) **times out at 60 s** — the type-index + direct-edge query is a >2,000× improvement on that access pattern.
4. **Class 8 is placement-insensitive**. `scannedIndex` is identical across all three DBs (313,407 for 8a/8b, 5,780 for 8c) and latency is within noise — because Design 2 duplicates *nodes* but **not edges**, so the type-index prunes the same edge slice regardless of placement. Label-wide aggregation is an edge-collection scan, not a sharded traversal.
5. **Cost scales with slice cardinality, not index quality**. All Class 8 queries are index-backed (`scannedFull = 0`); the time is the aggregation volume: 8c (5.8 K edges) ~20 ms, 8a/8b (313 K) ~0.7 s, and the dominant `discloses /ORG/ FIN_METRIC` slice (5.87 M edges, measured separately) ~11 s. Very large slices want a pre-aggregated rollup, not a bigger index.
6. **OneShard co-locates multi-hop execution**. 2-hop (5) **23.7 ms** → **2.0 ms** and the NL-grounding join (7) **4.9 ms** → **0.8 ms**, with the same global 1-shard graph (identical scan counts). The unpruned supernode traversal (4b) also improves **404** → **362 ms**.
7. **The documented SmartGraph trade-off shows up exactly where predicted**. *Global* concept access costs more: a `name` lookup for a shared concept returns **873 per-company copies** (scanIdx 28 → 873), and the cross-company 2-hop (5) crosses shards (**RemoteNode = 4** vs 0 on the baseline). Global concept roll-ups must aggregate `BY name` and pay cross-shard hops (multi-distribution-plan §5.7).

Caveats

- **Shared cluster, noisy latency**. Use `scannedIndex` and `RemoteNode` as the portable metrics; reserve wall-clock for a quiescent window before quoting absolute numbers.
- **† Smart temporal (query 6)**: the discovered `ticker (aa)` had no edges in 2022–2024, so that cell scans 0 — not a fair latency comparison. Pin a fixed high-volume ticker for a publishable temporal number.
- **Smart Class 8 grouping**: groups by `ticker`-prefixed `_from / _to` keys (per-company concept copies); since edges aren't duplicated the edge slice — and thus `scannedIndex` and latency — matches the baseline.

- `RemoteNode = 1 / GatherNode = 1` on most direct edge queries even on OneShard is just the coordinator shipping the final result from the single DBServer, *not* per-shard scatter/gather. The meaningful locality signals are the SmartGraph's per-company scan reduction (2, 3, 4a) and the `RemoteNode = 4` on the cross-company smart 2-hop (5).
-

History

- **v2.0 (2026-07-05)**: first cross-distribution run (Classes 1–7), pre-type-index. Established the ~250× SmartGraph supernode decomposition and the OneShard multi-hop win.
- **v1.0 (2026-07-05)**: single-shard baseline (Classes 1–7) on `FinReflectKG` — confirmed the VCI's narrow perfectly (`scannedFull = 0` throughout) and the supernode pruned-vs- unpruned gap (4a vs 4b). Superseded by the cross-distribution runs above.